Credit Risk Analysis: Predicting Loan Default

Introduction:

- Using a Dataset found on Kaggle titled “Credit_Risk_Analysis” we are to analyze information about loan applicants and their characteristics (such as age, income, intent) and their impact on the loan approval status (y variable). We have to develop a predictive model to determine the likelihood of loan approval status based on the X variables mentioned above, this will show us if a loan applicant’s status can be determined using variables such as Age, Income, Home Ownership Status, Employment Length in Years, Purpose of the loan, Amount of loan applied for, Interest rate on the loan, loan amount as a percentage of income, length of the applicant’s credit history and whether the loan has defaulted previously. And if we can use the given X variables to determine loan status which X variable is best for predicting? Given below is the research question having to do with this:

    - **Research Question:** Can we predict whether a loan applicant will default using their demographic, financial, and loan characteristics — and which of these factors are the strongest predictors of risk?

Data:

The Data used in this project was found from Kaggle titled ‘Credit_Risk_Analysis’. It has 12 columns and 32581 rows in which the columns represent the various X variables and the one Y variable and the rows represent all the applicants with things such as their age, income, etc. Below is a formal description of the variables (Taken from Kaggle):

- **ID**: Unique identifier for each loan applicant.
- **Age**: Age of the loan applicant.
- **Income**: Income of the loan applicant.
- **Home**: Home ownership status (Own, Mortgage, Rent).
- **Emp_Length**: Employment length in years.
- **Intent**: Purpose of the loan (e.g., education, home improvement).
- **Amount**: Loan amount applied for.
- **Rate**: Interest rate on the loan.
- **Status**: Loan approval status (Fully Paid, Charged Off, Current).
- **Percent_Income**: Loan amount as a percentage of income.
- **Default**: Whether the applicant has defaulted on a loan previously (Yes, No).
- **Cred_Length**: Length of the applicant's credit history.

The dependent variable of this dataset is ‘Status’, which has variable ‘Fully Paid’, ‘Charged Off’ and ‘Current’, in which we will convert Status into a binary variable with ‘0’ equaling ‘no default’ and ‘1’ equaling ‘Default, after which 78.2 percent of Statuses were ‘no-default’ and 21.8 percent Statuses were ‘default’. This is an imbalance which impacted modeling and evaluation choices in the project.

Preprocessing & Exploratory Data Analysis:

After reviewing the initial dataset and loading it into Google Colab, we renamed columns to more descriptive and specific names and got all the statistical info, after which we dropped the ‘ID’ column since it had nothing to do with determining the ‘Status’, our next step was seeing if there were any missing or duplicate values, where we found that there were 165 duplicate values as well as 4011 missing / null values (895 for Employment_in_years and 3116 in Interest_Rate).
We also checked for unrealistic values / outliers and found many rows where the employment in years was more than 60 and the age of the applicant was more than 100, we dropped these rows since they were impractical.
As for the 2011 missing / null values, we used imputation to fill in the gaps by using the KNNImputer where (k = 5) on standardized features rather than median imputation, this was done to keep relations between correlated variables.
There were also many datatype drifts caused by the scaling/imputation round-trip (many integer columns had become floats) and converted qualitative variables to appropriate types (this goes back to converting ‘Status’ into a binary variable having values of ‘0’ and ‘1’ only rather than ‘Fully Paid’, ‘Charged Off’ or ‘Current’.
After cleaning the data we were left with 11 columns and 31522 rows.

Feature Engineering:

One-hot encoded qualitative variables such as ‘Home_Ownership_Status’ and ‘Loan_Purpose’ with drop_first = True to avoid the dummy variable trap.
Tested an engineered ‘loan_to_income’ feature against the existing ‘Percent_Income’ column where we found a correlation of 0.999 which helped drop the redundant duplicate.
After encoding our final feature set was 16 columns.
We also did train_test_split where test_size was 0.2, random_state was 42 and ‘stratify = y’ to preserve class balance in both sets, after which we got a confirmation when Train target distribution and test target distribution had 0.784 distribution in status ‘0’ and 0.216 distribution in status ‘1’.
While the imbalanced distribution is at a questionable level, it is confirmed later that it is not a major issue because the lowest accuracy we had was 0.848 for Logistic Regression.

Modeling:

Given that we were going to see how different X variables such as Age, income, employment in years, interest rate, etc. were going to impact the Y variable or 'Status', we had to decide the model(s) based on what kind of variables we were dealing with, since we had qualitative variables as well as those with binary responses (0 or 1) we decided to use LightGBM and XGBoost which were in addition to Random Forest, Decision Tree and Logistic Regression. The tree based models were trained on unscaled features in which splits were threshold based rather than distance based, the only exception was Logistic Regression which required scaling given it was a distance-based model.

Logistic Regression helped with establishing that the problem was not linearly separable. While there was a low recall of 0.463, it was not something to worry about too much since it's a linear model and its underperformance relative to the tree-based models told you that default risk was NOT a simple weighted sum of features.

Decision Tree helped us get an interpretable first look at the model's actual logic. We were able to see this with an ROC-AUC of 0.887 as well as a high precision of 0.973, at the same time a lower recall showed an early version of the precision/recall tradeoff shown later with threshold tuning.

Random Forest helped us test whether ensembling alone fixed the Decision Tree's variance problem, we saw that averaging 200 trees improved ROC-AUC from 0.887 to 0.921 and recall improved from 0.646 to 0.648, mostly by reducing overfitting. While the improvement showed variance reduction helping, it still proved there was a meaningful gap to close which is what made us try to boost next than stop at bagging.

XGBoost helped us answer our research question, being the one that our threshold/interpretation work was built on, with the help of Gradient Boosting we were able to close any remaining performance gaps which helped the ROC-AUC jump to 0.942 and recall to 0.707. All the downstream analyses (threshold optimization, SHAP values and business recommendations) were built specifically on this model's outputs.

LightGBM helped confirm whether XGBoost's result was a coincidence of the specific algorithm. Performance was almost identical with ROC-AUC being 0.943 and recall being 0.706 which was using a different boosting implementation (leaf-wise growth compared to the level-wise growth of XGBoost) which confirmed the 0.94 ROC-AUC ceiling was not specific to one library's quirks but a real property of how much this dataset could help learn with gradient boosting.

Threshold Tuning:

Rather than accepting the 0.5 classification threshold, we optimized the classification threshold using a business cost framework (5:1 missed default-to-false-alarm ratio) which helped reduce missed defaulters by 47 percent at the optimal threshold, this was the case since missing a defaulter (false negative was likely costlier for a lender than a false alarm (false positive).

Results:

	Model	Accuracy	Precision	Recall	F1	ROC-AUC
4	LightGBM	0.928	0.947	0.706	0.809	0.943
3	XGBoost	0.928	0.948	0.707	0.810	0.942
2	Random Forest	0.918	0.957	0.648	0.772	0.921
1	Decision Tree	0.911	0.972	0.606	0.746	0.887
0	Logistic Regression	0.848	0.738	0.463	0.569	0.859

As shown, XGBoost was most helpful since it had the highest combination of accuracy, precision, recall, F1 and ROC-AUC, it was almost the same (in fact in some places less than) as LightGBM but the precision helped us consider it the most helpful. Logistic regression has a low recall of 0.463 which showed that the relationships in this data were meaningfully non-linear which explains why we used models like LightGBM and XGBoost even though we had easier options.

Threshold Optimization:

Even though at the optimized threshold the model misses 16% of actual defaulters (212 of 1362) where no threshold eliminates the tradeoff entirely, we were able to reduce missed defaulters by 47% at the optimal threshold where at the business optimized threshold of 0.18 the selected model caught 84.4% of actual defaulters (up to 70.7% of the default threshold) which reduced the missed defaults from 399 to 212 in the test set.

Feature Importance:

We used weight and gain-based importance (XGBoost native) to cross-validate the strongest predictors in which Home_Ownership_Status_RENT, Percent_Income and loan purpose categories ranked highest.
SHAP values were used for cross-validating the strongest predictors where Interest_Rate, Income and Percent_Income showed the most consistent impact on individual predictions across the test set.
The difference between methods spoke a lot in which split-based importance favored clean categorical splits (like renter status) while SHAP showed the overall influence of continuous financial variables. Both equally showed that demographic factors such as age and employment in years were unimportant relative to other variables.
From SHAP we saw that Higher Interest Rate resulted in Increased predicted default risk, Higher income decreased predicted default risk, higher loan-to-income ratio increased predicted default risk and renter status increased predicted default risk.

Conclusion:

Using the models and the statistical values for each of them, with XGBoost having a higher importance compared to other models we were able to conclude that the strongest predictors of default are financial strain indicators such as loan-to-income ratios, interest rate and income rather than purely demographic variables like age or employment length which ranked low regardless of the method. This suggests that default risk is driven more by an applicant's current financial capacity compared to their loan burden than my static demographic traits.

Recommendation:

A lender should consider variables such as weight loan-to-income ratio and interest rate when wanting to derive a conclusion for such applications and pair the model's probability output with the threshold of 0.18 to highlight applications with high risk for thorough review before using a machine to make conclusions about it.

Limitations:

Cost Assumption: The 5:1 missed default-to-false-alarm cost ratio used for threshold tuning shows visualizations not derived from real financial data.
No temporal validation: The train/test split is random, not time-based. In reality a lender would validate on more recent applications to check for changing economic conditions affecting default patterns.
Simplified Feature Set: The dataset uses a reduced set of features compared to what a real underwriting model should have (ex: credit score, existing debt obligations, etc).
Residual Risk: Even at the optimized threshold the model misses 16% of actual defaulters (212 of 1362) which shows that no threshold eliminated the tradeoff fully and only shifted where the balance sat.

Resources:

- https://www.kaggle.com/datasets/nanditapore/credit-risk-analysis
- scikit-learn: machine learning in Python — scikit-learn 1.9.0 documentation

AI Acknowledgement:

- Claude was used to write code for visualizations as well as help with any kind of explanation needed for any code/output, it also helped write comments to explain what the code was about at a certain point.

	Id	Age	Income	Home	Emp_length	Intent	Amount	Rate	Status	Percent_income	Default	Cred_length
0	0	22	59000	RENT	123.0	PERSONAL	35000	16.02	1	0.59	Y	3
1	1	21	9600	OWN	5.0	EDUCATION	1000	11.14	0	0.10	N	2
2	2	25	9600	MORTGAGE	1.0	MEDICAL	5500	12.87	1	0.57	N	3
3	3	23	65500	RENT	4.0	MEDICAL	35000	15.23	1	0.53	N	2
4	4	24	54400	RENT	8.0	MEDICAL	35000	14.27	1	0.55	Y	4
...	...	...	...	...	...	...	...	...	...	...	...	...
32576	32576	57	53000	MORTGAGE	1.0	PERSONAL	5800	13.16	0	0.11	N	30
32577	32577	54	120000	MORTGAGE	4.0	PERSONAL	17625	7.49	0	0.15	N	19
32578	32578	65	76000	RENT	3.0	HOMEIMPROVEMENT	35000	10.99	1	0.46	N	28
32579	32579	56	150000	MORTGAGE	5.0	PERSONAL	15000	11.48	0	0.10	N	26
32580	32780	66	42000	RENT	2.0	MEDICAL	6475	9.99	0	0.15	N	30

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 32581 entries, 0 to 32580
Data columns (total 12 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   Id                      32581 non-null  int64
 1   Age                     32581 non-null  int64
 2   Income                  32581 non-null  int64
 3   Home_Ownership_Status   32581 non-null  object
 4   Employment_In_Years     31686 non-null  float64
 5   Loan_Purpose            32581 non-null  object
 6   Loan_Amount_Applied     32581 non-null  int64
 7   Interest_Rate           29465 non-null  float64
 8   Status                  32581 non-null  int64
 9   Percent_income          32581 non-null  float64
 10  Previous_Loan_Default   32581 non-null  object
 11  Credit_History_Length   32581 non-null  int64
dtypes: float64(3), int64(6), object(3)
memory usage: 3.0+ MB
```

]:
```
# ID is irrelevant for analyzing or making conclusions from here, therefore it is dropped.
df.drop(columns=['Id'], inplace=True, errors='ignore')
```

]:
```
# Getting the count of duplicated variables
df.duplicated().sum()
```

]:
```
np.int64(165)
```

]:
```
# Knowing how many null values there are and where, and what percent they make up.
missing = df.isnull().sum()
missing_pct = (missing / len(df) * 100).round(2)
pd.DataFrame({'Missing Count': missing, 'Missing %': missing_pct}).query('`Missing Count` > 0')
```

]:

	Missing Count	Missing %
Employment_In_Years	895	2.75
Interest_Rate	3116	9.56

In [76]:
```
# Know what percentage of people got no (or 0) to status and yes (or 1).
df['Status'].value_counts(normalize=True).round(3)
```

Out[76]:

Status	proportion
0	0.782
1	0.218

dtype: float64

In [77]:
```
# Know how many rows and columns we are working with
df.shape
```

Out[77]: (32581, 11)

STEP TWO: Data Cleaning

In [78]:
```
# Seeing possible unrealistic values for age
df[df['Age'] > 100][['Age', 'Employment_In_Years', 'Income', 'Status']]
```

Out[78]:

	Age	Employment_In_Years	Income	Status
81	144	4.0	250000	0
183	144	4.0	200000	0
575	123	2.0	80004	0
747	123	7.0	78000	0
32297	144	12.0	6000000	0

In [79]:
```
# Seeing possible unrealistic values for employment in years.
df[df['Employment_In_Years'] > 60][['Age', 'Employment_In_Years', 'Income', 'Status']]
```

Out[79]:

	Age	Employment_In_Years	Income	Status
0	22	123.0	59000	1
210	21	123.0	192000	0

In [80]:
```
# Remove implausible ages (>100) and implausible employment lengths (>60 years)
before = df.shape[0]
df = df[(df['Age'] <= 100) & (df['Employment_In_Years'] <= 60)]
after = df.shape[0]
print(f"Removed {before - after} rows (({(before-after)/before*100:.2f}% of data)")
```

```
Removed 902 rows (2.77% of data)
```

In [82]:
```
# Show total null values for employment in years and interest rate as well as describe both variables
df[['Interest_Rate', 'Employment_In_Years']].isnull().sum()
df[['Interest_Rate', 'Employment_In_Years']].describe()
```

Out[82]:

	Interest_Rate	Employment_In_Years
count	31679.000000	31679.000000
mean	11.037431	4.782064
std	3.109820	4.034848
min	5.420000	0.000000
25%	8.490000	2.000000
50%	10.990000	4.000000
75%	13.220000	7.000000
max	23.220000	41.000000

In [83]:
```
# Get the data types of all variables
df.dtypes
```

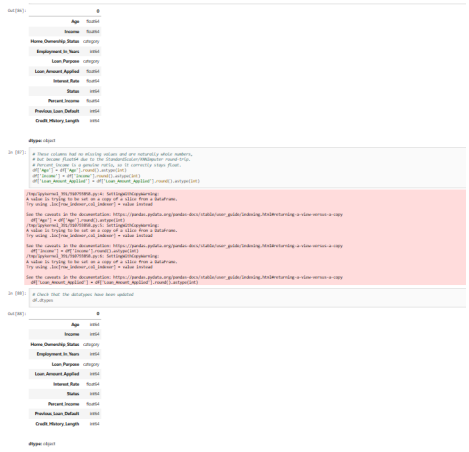
Out[83]:

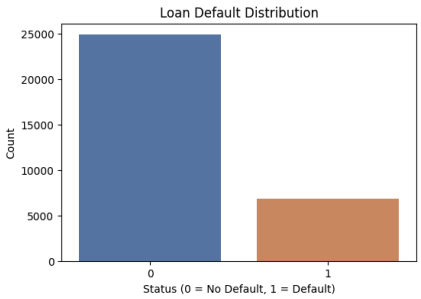
	0
Age	float64
Income	float64
Home_Ownership_Status	object
Employment_In_Years	float64
Loan_Purpose	object
Loan_Amount_Applied	float64
Interest_Rate	float64
Status	int64
Percent_Income	float64
Previous_Loan_Default	object
Credit_History_Length	float64

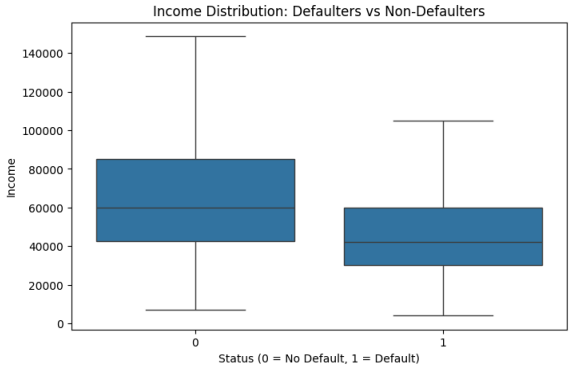
dtype: object

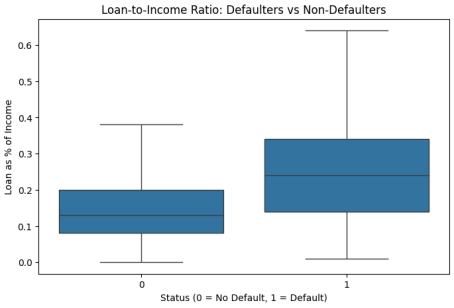
In [84]:
```
# Show how many unique values there are for 'object' variables and the 'Status' variable which is binary
for col in ['Home_Ownership_Status', 'Loan_Purpose', 'Previous_Loan_Default', 'Status']:
    print(f"{col}: {df[col].unique()}")
```

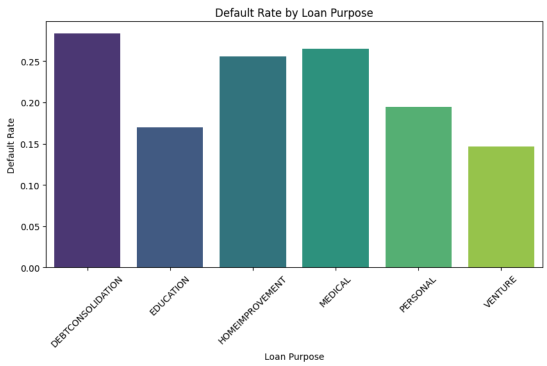
```
Home_Ownership_Status: ['OWN' 'MORTGAGE' 'RENT' 'OTHER']
Loan_Purpose: ['EDUCATION' 'MEDICAL' 'VENTURE' 'PERSONAL' 'HOMEIMPROVEMENT'
 'DEBTCONSOLIDATION']
Previous_Loan_Default: ['N' 'Y']
Status: [0 1]
```

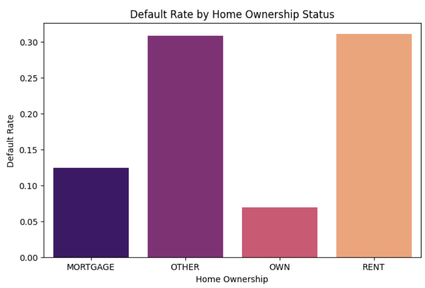



Loan Default Distribution
Count
Status (0 = No Default, 1 = Default)


Income Distribution: Defaulters vs Non-Defaulters
Income
Status (0 = No Default, 1 = Default)


Loan-to-Income Ratio: Defaulters vs Non-Defaulters
Loan as % of Income
Status (0 = No Default, 1 = Default)

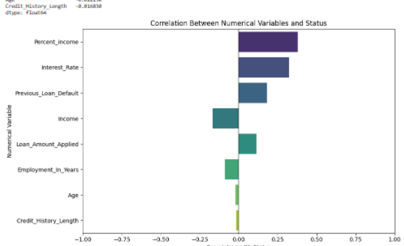




In [94]:

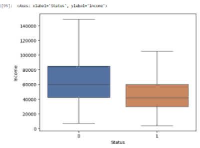
```python
default_numeric = df['Status']

# Get all numerical X variables, excluding ID
numerical_cols = df.drop(columns=['Id', 'Status'], errors='ignore') \
                  .select_dtypes(include=['int64', 'float64']).columns

# Calculate correlation with Default
correlations = df[numerical_cols].corrwith(default_numeric)

# Sort by absolute correlation
correlations = correlations.reindex(
    correlations.abs().sort_values(ascending=False).index
)

# Print correlations to verify data
print("Correlations:")
print(correlations)

# Plot
plt.figure(figsize=(10, 6))

sns.barplot(
    x=correlations.values,
    y=correlations.index,
    palette=sns.color_palette("viridis", n_colors=len(correlations)) # Assign a unique color to each bar
)

plt.axvline(0, color='black', linewidth=1)
plt.xlim(-1, 1) # Set x-axis limits from -1 to 1

plt.title('Correlation Between Numerical Variables and Status')
plt.xlabel('Correlation with Status')
plt.ylabel('Numerical Variable')

plt.tight_layout()
plt.show()
```

```
/tmp/ipykernel_391/1598199119.py:22: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

  sns.barplot(
Correlations:
Percent_Income          0.379823
Interest_Rate           0.324579
Previous_Loan_Default   0.181098
Income                 -0.164128
Loan_Amount_Applied     0.112188
Employment_In_Years    -0.085966
Age                    -0.022138
Credit_History_Length  -0.016838
dtype: float64
```

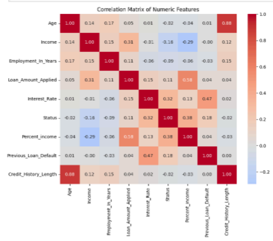


In [95]:
```python
sns.boxplot(data=df, x='Status', y='Income', hue='Status',
            palette=['#4C72B0', '#DD8452'], legend=False, showfliers=False)
```

Out[95]: <Axes: xlabel='Status', ylabel='Income'>



In [96]:
```python
# Correlation heatmap (numeric features only)
plt.figure(figsize=(9,7))
numeric_df = df.select_dtypes(include=['int64', 'float64'])
corr = numeric_df.corr()
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm', center=0)
plt.title('Correlation Matrix of Numeric Features')
plt.show()
```



In [97]:
```python
print("Correlation with Status:")
print(f"Age: {df['Age'].corr(df['Status']):.3f}")
print(f"Credit_History_Length: {df['Credit_History_Length'].corr(df['Status']):.3f}")
```

```
Correlation with Status:
Age: -0.022
Credit_History_Length: -0.017
```

In [98]:
```python
categorical_cols = ['Home_Ownership_Status', 'Loan_Purpose']

for col in categorical_cols:
    print(f"\n{col}")
    print(df[col].value_counts())
```

```
Home_Ownership_Status
Home_Ownership_Status
RENT        16072
MORTGAGE    13090
OWN          2410
OTHER         107
Name: count, dtype: int64

Loan_Purpose
Loan_Purpose
EDUCATION            6288
MEDICAL              5897
VENTURE              5553
PERSONAL             5367
DEBTCONSOLIDATION    5064
HOMEIMPROVEMENT      3510
Name: count, dtype: int64
```

In [99]:
```python
df.columns.tolist()
```

Out[99]:
```
['Age',
 'Income',
 'Home_Ownership_Status',
 'Employment_In_Years',
 'Loan_Purpose',
 'Loan_Amount_Applied',
 'Interest_Rate',
 'Status',
 'Percent_Income',
 'Previous_Loan_Default',
 'Credit_History_Length']
```

In [100…
```python
print(df.dtypes)
```

```
Age                         int64
Income                      int64
Home_Ownership_Status    category
Employment_In_Years         int64
Loan_Purpose             category
Loan_Amount_Applied         int64
Interest_Rate             float64
Status                      int64
Percent_Income            float64
Previous_Loan_Default       int64
Credit_History_Length       int64
dtype: object
```

In [101…
```python
print(df.columns.tolist())
```

```
['Age', 'Income', 'Home_Ownership_Status', 'Employment_In_Years', 'Loan_Purpose', 'Loan_Amount_Applied', 'Interest_Rate', 'Status', 'Percent_Income', 'Previous_Loan_Default', 'Credit_History_Length']
```

In [102…
```python
# FIX
df['loan_to_income'] = (df['Loan_Amount_Applied'] / df['Income'].replace(0, pd.NA)).round(2)

print(df[['Percent_Income', 'loan_to_income']].corr())
print("Missing values created:", df['loan_to_income'].isnull().sum())
```

```
                Percent_Income  loan_to_income
Percent_Income        1.000000        0.999179
loan_to_income        0.999179        1.000000
Missing values created: 0
```

```
/tmp/ipykernel_391/4035187027.py:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  df['loan_to_income'] = (df['Loan_Amount_Applied'] / df['Income'].replace(0, pd.NA)).round(2)
```

In [103…
```python
df = df.drop(columns=['loan_to_income'])
```

In [104…
```python
print(df[['Loan_Amount_Applied', 'Income']].head())
```

```
   Loan_Amount_Applied  Income
1                 1000    9600
2                 5500    9600
3                35000   65500
4                35000   54400
5                 2500    9900
```

In [118...

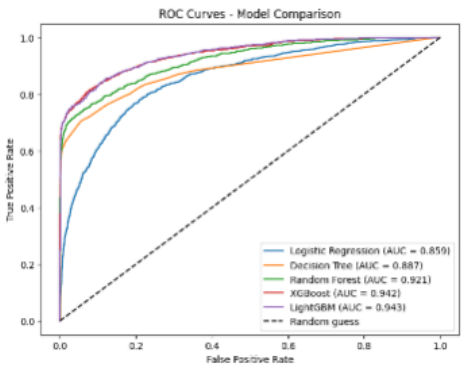
```
# Store the true labels, predicted classes, and predicted probabilities for each model.
# This dictionary allows for easy iteration and evaluation of all models.
results = {
    'Logistic Regression': (y_test, log_reg_pred, log_reg_proba),
    'Decision Tree': (y_test, dt_pred, dt_proba),
    'Random Forest': (y_test, rf_pred, rf_proba),
    'XGBoost': (y_test, xgb_pred, xgb_proba),
    'LightGBM': (y_test, lgbm_pred, lgbm_proba)
}

# Initialize an empty list to store performance metrics for each model.
comparison = []
# Iterate through each model's results to calculate and store metrics.
for name, (y_true, y_pred, y_proba) in results.items():
    comparison.append({
        'Model': name,
        'Accuracy': accuracy_score(y_true, y_pred),
        'Precision': precision_score(y_true, y_pred),
        'Recall': recall_score(y_true, y_pred),
        'F1': f1_score(y_true, y_pred),
        'ROC-AUC': roc_auc_score(y_true, y_proba)
    })

# Convert the list of dictionaries into a Pandas DataFrame for better readability and sorting.
comparison_df = pd.DataFrame(comparison).sort_values('ROC-AUC', ascending=False)
# Display the comparison table, rounding metrics to 3 decimal places.
comparison_df.round(3)
```

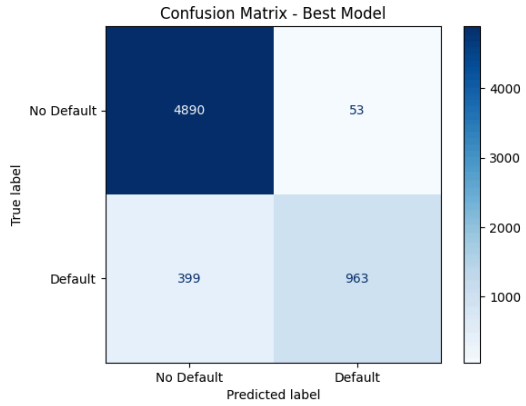
Out[118...

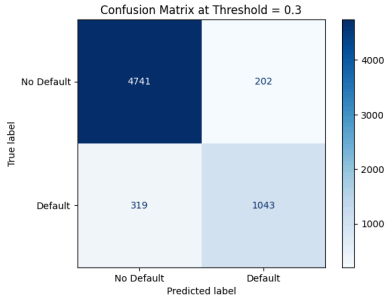
	Model	Accuracy	Precision	Recall	F1	ROC-AUC
4	LightGBM	0.928	0.947	0.706	0.809	0.943
3	XGBoost	0.928	0.948	0.707	0.810	0.942
2	Random Forest	0.918	0.957	0.648	0.772	0.921
1	Decision Tree	0.911	0.972	0.606	0.746	0.887
0	Logistic Regression	0.848	0.738	0.463	0.569	0.859

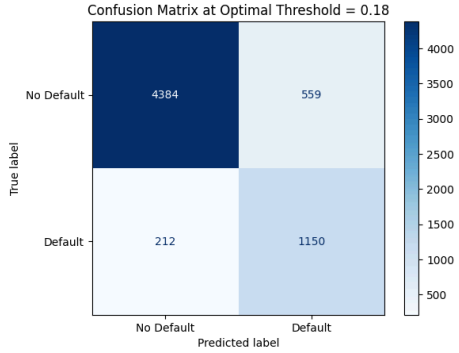
In [119...

```
# Create a new figure for the ROC curve plot.
plt.figure(figsize=(8, 6))
# Iterate through each model to plot its ROC curve.
for name, (y_true, y_pred, y_proba) in results.items():
    # Calculate False Positive Rate (fpr), True Positive Rate (tpr), and thresholds for the ROC curve.
    fpr, tpr, _ = roc_curve(y_true, y_proba)
    # Calculate the Area Under the Receiver Operating Characteristic Curve (ROC-AUC).
    auc_score = roc_auc_score(y_true, y_proba)
    # Plot the ROC curve for the current model, including its AUC score in the label.
    plt.plot(fpr, tpr, label=f'{name} (AUC = {auc_score:.3f})')

# Plot the diagonal dashed line representing a random guess classifier.
plt.plot([0, 1], [0, 1], 'k--', label='Random guess')
# Label the x-axis as 'False Positive Rate'.
plt.xlabel('False Positive Rate')
# Label the y-axis as 'True Positive Rate'.
plt.ylabel('True Positive Rate')
# Set the title of the plot.
plt.title('ROC Curves - Model Comparison')
# Display the legend to identify each model's curve.
plt.legend()
# Show the plot.
plt.show()
```









In [122...

```
# Define a function to evaluate model metrics at a given classification threshold.
def evaluate_at_threshold(y_true, y_proba, threshold):
    # Convert probabilities to binary predictions based on the specified threshold.
    y_pred_thresh = (y_proba >= threshold).astype(int)
    # Return a dictionary containing Precision, Recall, F1-score, and Accuracy at this threshold.
    return {
        'Threshold': threshold,
        'Precision': precision_score(y_true, y_pred_thresh),
        'Recall': recall_score(y_true, y_pred_thresh),
        'F1': f1_score(y_true, y_pred_thresh),
        'Accuracy': accuracy_score(y_true, y_pred_thresh)
    }

# Calculate evaluation metrics for XGBoost at a set of predefined thresholds (0.5, 0.4, 0.3, 0.2, 0.15).
threshold_results = [evaluate_at_threshold(y_test, xgb_proba, t)
                     for t in [0.5, 0.4, 0.3, 0.2, 0.15]]

# Display the results in a DataFrame, rounded to 3 decimal places.
pd.DataFrame(threshold_results).round(3)
```

Out[122...

	Threshold	Precision	Recall	F1	Accuracy
0	0.50	0.948	0.707	0.810	0.928
1	0.40	0.914	0.731	0.812	0.927
2	0.30	0.838	0.766	0.800	0.917
3	0.20	0.709	0.827	0.764	0.889
4	0.15	0.616	0.867	0.721	0.855

In [124...

```
# Define a function to calculate the business cost based on false negatives and false positives.
# cost_fn: cost of a false negative (missing an actual defaulter).
# cost_fp: cost of a false positive (incorrectly classifying a non-defaulter as a defaulter).
def business_costly_true, y_proba, threshold, cost_fn=1, cost_fp=1):
    # Convert probabilities to binary predictions using the given threshold.
    y_pred = (y_proba >= threshold).astype(int)
    # Compute the confusion matrix.
    cm = confusion_matrixCosty_true, y_pred)
    # Extract True Negatives, False Positives, False Negatives, and True Positives.
    tn, fp, fn, tp = cm.ravel()
    # Calculate the total business cost.
    total_cost = (fn * cost_fn) + (fp * cost_fp)
    return total_cost

# Initialize an empty list to store cost results for various thresholds.
cost_results = []
# Iterate through a set of predefined thresholds.
for t in [0.5, 0.4, 0.3, 0.2, 0.15]:
    # Calculate the business cost for each threshold.
    cost = business_costly_test, xgb_proba, t)
    # Append the threshold and its estimated cost to the results list.
    cost_results.append({'Threshold': t, 'Estimated Cost': cost})

# Convert the list of dictionaries to a DataFrame and display it.
pd.DataFrame(cost_results)
```

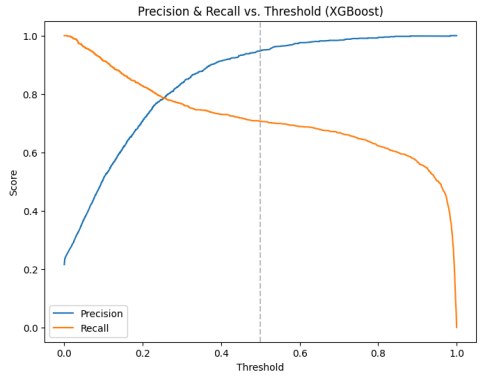
Out[124...

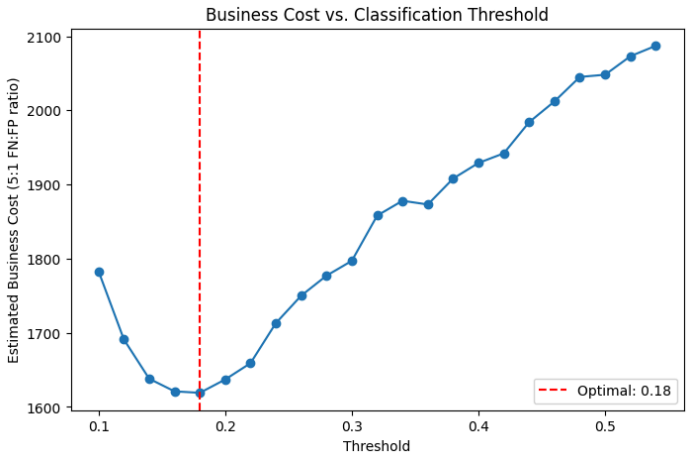
	Threshold	Estimated Cost
0	0.50	2048
1	0.40	1929
2	0.30	1797
3	0.20	1657
4	0.15	1640

In [125...

```
# Define a finer range of thresholds for more granular cost analysis.
finer_thresholds = np.arange(0.10, 0.55, 0.02)

# Initialize an empty list to store cost results for the finer thresholds.
cost_results_fine = []
# Iterate through the finer thresholds.
for t in finer_thresholds:
    # Calculate the business cost for the current threshold.
    cost = business_costly_test, xgb_proba, t)
    # Append the rounded threshold and its estimated cost to the results list.
    cost_results_fine.append({'Threshold': round(t, 2), 'Estimated Cost': cost})

# Convert the list of dictionaries to a DataFrame.
cost_df = pd.DataFrame(cost_results_fine)
# Print the DataFrame containing costs for the finer thresholds.
print(cost_df)

# Find the row in the cost DataFrame that corresponds to the minimum estimated cost.
best_row = cost_df.loc[cost_df['Estimated Cost'].idxmin()]
# Print the optimal threshold and its corresponding cost.
print(f"\nOptimal threshold: {best_row['Threshold']} (cost = {best_row['Estimated Cost']:.0f})")
```

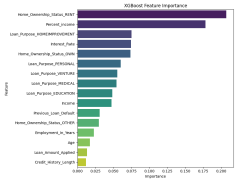
```
    Threshold  Estimated Cost
0        0.10            1781
1        0.12            1693
2        0.14            1638
3        0.16            1623
4        0.18            1619
5        0.20            1637
6        0.22            1659
7        0.24            1713
8        0.26            1750
9        0.28            1777
10       0.30            1797
11       0.32            1858
12       0.34            1878
13       0.36            1873
14       0.38            1908
15       0.40            1929
16       0.42            1942
17       0.44            1984
18       0.46            2014
19       0.48            2045
20       0.50            2048
21       0.52            2073
22       0.54            2087

Optimal threshold: 0.18 (cost = 1619)
```





Metric	Default (0.5)	Optimal (0.18)
Recall	0.707	0.844
Precision	0.948	0.674
Missed defaulters	399	212
False alarms	55	599
Estimated cost	2048	1610

```
# Create Dataframe of feature importances
importances = pd.DataFrame({
    'Feature': X_train.columns,
    'Importance': xgb_model.feature_importances_
}).sort_values('Importance', ascending=False)

# Plot the feature importance of all X values for the XGBoost model
plt.figure(figsize=(9, 7))
sns.barplot(data=importances, x='Importance', y='Feature', palette='viridis')
plt.title('XGBoost Feature Importance')
plt.xlabel('Importance')
plt.tight_layout()
plt.show()

# Display the importances in a table format
importances
```



Out[128...

	Feature	Importance
10	Home_Ownership_Status_RENT	0.207029
5	Percent_Income	0.178007
12	Loan_Purpose_HOMEIMPROVEMENT	0.075186
4	Interest_Rate	0.074305
9	Home_Ownership_Status_OWN	0.073907
14	Loan_Purpose_PERSONAL	0.060281
15	Loan_Purpose_VENTURE	0.055245
13	Loan_Purpose_MEDICAL	0.054455
11	Loan_Purpose_EDUCATION	0.048291
1	Income	0.047527
6	Previous_Loan_Default	0.030857
8	Home_Ownership_Status_OTHER	0.029747
2	Employment_In_Years	0.022829
0	Age	0.017329
3	Loan_Amount_Applied	0.013376
7	Credit_History_Length	0.011630

In [129...

```
# Create the XGBClassifier model
xgb_model_gain = XGBClassifier(
    n_estimators=200, max_depth=6, learning_rate=0.1,
    random_state=42, eval_metric='logloss', importance_type='gain'
)

# Fit the model
xgb_model_gain.fit(X_train, y_train)

# Extract feature importance by gain
importances_gain = pd.DataFrame({
    'Feature': X_train.columns,
    'Importance (Gain)': xgb_model_gain.feature_importances_
}).sort_values('Importance (Gain)', ascending=False)

# Display below
importances_gain
```

Out[129...

	Feature	Importance (Gain)
10	Home_Ownership_Status_RENT	0.207029
5	Percent_Income	0.178007
12	Loan_Purpose_HOMEIMPROVEMENT	0.075186
4	Interest_Rate	0.074305
9	Home_Ownership_Status_OWN	0.073907
14	Loan_Purpose_PERSONAL	0.060281
15	Loan_Purpose_VENTURE	0.055245
13	Loan_Purpose_MEDICAL	0.054455
11	Loan_Purpose_EDUCATION	0.048291
1	Income	0.047527
6	Previous_Loan_Default	0.030857
8	Home_Ownership_Status_OTHER	0.029747
2	Employment_In_Years	0.022829
0	Age	0.017329
3	Loan_Amount_Applied	0.013376
7	Credit_History_Length	0.011630

In [136...

```
# Create the explainer
explainer = shap.TreeExplainer(xgb_model)

# Compute Shap values for the dataset
shap_values = explainer.shap_values(X_test)

# Visualize feature importance
shap.summary_plot(shap_values, X_test, show=True)
```